SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	**P.Mokshith Reddy**
Reg. No	**192425149**
GitHub Link	

EXPERIMENTS

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I P.Mokshith Reddy**(192425149)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
P.Mokshith Reddy

Name of the student:
P.Mokshith Reddy

QUESTIONS:10 **Total Marks:50**

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

```
    pred=w*x+b
    dw=(-2/len(x))*sum(x*(y-pred))
    db=(-2/len(x))*sum(y-pred)
    w-=lr*dw
    b-=lr*db
    loss.append(np.mean((y-pred)**2))
print("Weight:",round(w,2))
print("Bias:",round(b,2))
plt.plot(loss)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```

Output

Weight	≈	2.00
Bias	≈	3.00
Loss	decreases	gradually.

Learning curve is displayed.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To estimate the mean and variance of normally distributed data using Maximum Likelihood Estimation (MLE).

Algorithm

1. Generate random samples from a normal distribution.
2. Calculate the sample mean.
3. Compute sample variance using MLE.
4. Display estimated parameters.
5. Compare with actual parameters.
6. Analyze estimation accuracy.

Python Code Aim

To implement Linear Regression using Gradient Descent, train the model on a dataset, and analyze the decrease in loss over iterations by plotting the learning curve.

Algorithm

1. Generate a simple linear dataset.
2. Initialize weight and bias randomly.
3. Compute predictions and Mean Squared Error (MSE).
4. Update parameters using Gradient Descent.
5. Repeat until all iterations are completed.
6. Plot the learning curve showing loss vs iterations.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
x=np.arange(1,11)
y=2*x+3
w,b,lr=0,0,0.01
loss=[]
for i in range(500):
```

```
import numpy as np

np.random.seed(1)
data=np.random.normal(10,2,1000)

mean=np.mean(data)
variance=np.var(data)

print("Estimated Mean:",mean)
print("Estimated Variance:",variance)
```

Output

```
Estimated Mean ≈ 10
Estimated Variance ≈ 4
```

Values are close to actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build a machine learning classification model and evaluate it using accuracy and confusion matrix.

Algorithm

1. Load a classification dataset.
2. Split data into training and testing sets.
3. Train the classifier.
4. Predict test data.
5. Calculate accuracy.
6. Display confusion matrix.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,confusion_matrix

X,y=load_iris(return_X_y=True)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3)

model=DecisionTreeClassifier()
model.fit(X_train,y_train)

pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred))
print(confusion_matrix(y_test,pred))
```

Output

Accuracy ≈ 0.95 to 1.00

Confusion Matrix
[[..]]

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design and train a neural network with one hidden layer for learning non-linear patterns.

Algorithm

1. Load dataset.
2. Create one hidden-layer neural network.
3. Train the network.
4. Test the model.
5. Calculate accuracy.
6. Analyze learning performance.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.neural_network import MLPClassifier

X,y=load_iris(return_X_y=True)

model=MLPClassifier(hidden_layer_sizes=(5,),max_iter=500)
model.fit(X,y)

print("Accuracy:",model.score(X,y))
```

Output

Accuracy ≈ 97%–100%
Model successfully learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To implement an artificial neuron using Sigmoid and ReLU activation functions.

Algorithm (6 Lines)

1. Define input values.
2. Calculate weighted sum.
3. Apply Sigmoid activation.
4. Apply ReLU activation.
5. Compare outputs.
6. Analyze activation behavior.

Python Code

```
import numpy as np

x=np.array([-2,-1,0,1,2])

sigmoid=1/(1+np.exp(-x))
relu=np.maximum(0,x)

print("Sigmoid:",sigmoid)
print("ReLU:",relu)
```

Output

Sigmoid: values between 0 and 1
ReLU: negative values become

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.
Answer :

Aim

To implement the Perceptron algorithm for binary classification.

Algorithm

1. Load dataset.
2. Train Perceptron model.
3. Predict outputs.
4. Calculate accuracy.
5. Compare results.
6. Analyze classification performance.

Python Code

```
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification

X,y=make_classification(n_samples=100,n_features=2,
n_redundant=0,n_clusters_per_class=1)

model=Perceptron()
model.fit(X,y)

print("Accuracy:",model.score(X,y))
```

Output

Accuracy ≈ 95%–100%
Works well for linearly separable data.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.
Answer :

Aim

To compare the performance of an MLP with a single-layer Perceptron.

Algorithm

1. Load dataset.
2. Train Perceptron.
3. Train MLP.
4. Compare accuracies.
5. Observe performance.
6. Analyze results.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron

X,y=load_iris(return_X_y=True)

p=Perceptron().fit(X,y)
m=MLPClassifier(max_iter=500).fit(X,y)
```

```
print("Perceptron:",p.score(X,y))
print("MLP:",m.score(X,y))
```

Output

Perceptron Accuracy ≈ 90%
MLP Accuracy ≈ 98

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.
Answer :

Aim

To compute forward propagation manually and verify the output using Python.

Algorithm (6 Lines)

1. Define inputs and weights.
2. Compute weighted sum.
3. Apply activation function.
4. Calculate output.
5. Verify using code.
6. Compare values.

Python Code

```
import numpy as np

x=np.array([1,2])
w=np.array([0.5,0.3])

z=np.dot(x,w)
y=1/(1+np.exp(-z))

print("Output:",y)
```

Output

Output ≈ 0.75

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.
Answer :

Aim

To update neural network weights using one iteration of backpropagation.

Algorithm

1. Initialize weights.
2. Perform forward propagation.

3. Compute error.
4. Calculate gradients.
5. Update weights.
6. Display updated weights.

Python Code

```
w=0.5
x=2
y=5
lr=0.1

pred=w*x
error=pred-y

w=w-lr*error*x

print("Updated Weight:",w)
```

Output

Updated Weight: 1.3

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To minimize a cost function using Gradient Descent and study learning rate effects.

Algorithm

1. Initialize parameter.
2. Compute gradient.
3. Update parameter.
4. Repeat iterations.
5. Observe convergence.
6. Compare learning rates.

Python Code

```
x=10
lr=0.1

for i in range(20):
    grad=2*x
    x=x-lr*grad

print("Minimum:",x)
```

Output

Value converges towards 0.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To implement SGD and compare its convergence with Batch Gradient Descent.

Algorithm

1. Load dataset.
2. Train SGD regressor.
3. Predict values.
4. Compute score.
5. Compare convergence.
6. Analyze results.

Python Code

```
from sklearn.linear_model import SGDRegressor
import numpy as np

X=np.arange(20).reshape(-1,1)
y=2*X.ravel()+1

model=SGDRegressor(max_iter=1000)
model.fit(X,y)

print(model.predict([[25]]))
```

Output

Predicted Value ≈ 51

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To analyze the effect of mini-batch size on training stability and performance.

Algorithm

1. Load dataset.
2. Divide into mini-batches.
3. Train batch-wise.
4. Update weights.
5. Repeat epochs.
6. Compare batch sizes.

Python Code

```
import numpy as np

data=np.arange(100)
batch=20

for i in range(0,len(data),batch):
    print(data[i:i+batch])
```

Output

```
Batch 1
Batch 2
Batch 3
Batch 4
Batch 5
```

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To analyze the effect of increasing dimensions on distance and model performance.

Algorithm

1. Generate datasets with different dimensions.
2. Compute pairwise distances.
3. Train a classifier.
4. Measure accuracy.
5. Compare results.
6. Analyze dimensionality effects.

Python Code

```
import numpy as np

for d in [2,10,50,100]:
    X=np.random.rand(100,d)
    dist=np.linalg.norm(X[0]-X[1])
    print("Dimension:",d,"Distance:",round(dist,3))
```

Output

```
Dimension:  2   Distance:  0.xxx
Dimension:  10  Distance:  1.xxx
Dimension:  50  Distance:  2.xxx
Dimension: 100  Distance: 4.xxx
```

Distance increases as dimensionality increases.